

K-Nearest Neighbor (k-NN) in Vector Databases

Karina Leśkiewicz

March 2026

- The k-Nearest Neighbors (k-NN) algorithm is a non-parametric, lazy learning method used for both classification and regression.
- Unlike eager learners, k-NN does not derive a discriminative function from the training data but instead stores the entire dataset, postponing computation until the inference phase.

Similarity Hypothesis & Outcomes

- The core principle relies on the **similarity hypothesis**: data points with similar properties tend to cluster in the same region of the feature space.
- For a given query point, the algorithm identifies the k closest observations and determines the outcome via:
 - **Majority Voting** (for classification)
 - **Mean/Average** (for regression)

Hyperparameter Tuning: Determining k

- The selection of k is a critical step.
- A small k makes the model sensitive to noise (high variance/overfitting), while a large k may blur the decision boundaries between classes (high bias/underfitting).

1. Cross-Validation

- To find the optimal k , we typically use **m -fold cross-validation**.
- The model is tested m times, each time using a different subset as the validation set and the remaining $m - 1$ parts as training data.
- The k value that yields the highest average accuracy is selected.

2. Empirical Optimization (The Elbow Method)

- In this approach, we plot the error rate as a function of k .
- The "elbow" of the curve — where the decrease in error levels off — indicates the optimal balance between model complexity and generalization.

Distance Metrics (Norms) in Vector Space

- k-NN relies on distance metrics to identify the proximity of data points.
- In vector databases, these are often referred to as **norms**.

Manhattan & Euclidean Distances

Manhattan Distance (L_1 Norm)

- Also known as "Taxicab" distance. It calculates the distance as the sum of absolute differences along each axis.

$$d(x, y) = \sum_{i=1}^n |x_i - y_i| \quad (1)$$

- Often more robust than L_2 in high-dimensional spaces.

Euclidean Distance (L_2 Norm)

- The most common metric, representing the "straight-line" distance between two points.

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2)$$

- Sensitive to outliers due to squaring, but standard for most physical/geometric applications.

- The Minkowski distance is a generalized metric:

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p} \quad (3)$$

- It provides flexibility:
 - For $p = 1$, it is equivalent to the **Manhattan distance**.
 - For $p = 2$, it is equivalent to the **Euclidean distance**.

Cosine Similarity

- Cosine Similarity measures the orientation (angle) between two vectors, regardless of their magnitude.
- Crucial in NLP because it focuses on the "direction" of the meaning rather than the length of the document.

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (4)$$

- In vector databases, the **Cosine Distance** is often defined as $1 - \text{Cosine Similarity}$.

Approximate k-NN Search (ANN)

- Approximate means that for a given search, the neighbors returned are an estimate of the true k-nearest neighbors.
- This method offers the best search scalability for large data sets.

Locality Sensitive Hashing (LSH)

- The goal of LSH is to hash input items so that similar items map to the same "buckets" with high probability.
- **Mathematical Principle:** A family of functions $\mathcal{H}$ is locality-sensitive if:
 - If $\text{dist}(x, y) \leq R$, then $P[h(x) = h(y)] \geq p_1$
 - If $\text{dist}(x, y) \geq cR$, then $P[h(x) = h(y)] \leq p_2$
- Random Projections uses a random unit vector w :

$$h_w(x) = \text{sign}(w \cdot x) \quad (5)$$

- This effectively partitions the space with hyperplanes.

LSH for Different Norms

LSH functions must be specifically designed for the distance metric they intend to approximate.

1. LSH for L_2 (Euclidean)

- Uses **p -stable distributions** (Gaussian). Vector is divided into "bins" of width w :

$$h_{a,b}(v) = \left\lfloor \frac{a \cdot v + b}{w} \right\rfloor \quad (6)$$

2. LSH for L_1 (Manhattan)

- Random vector a is sampled from a **Cauchy distribution** (1-stable, consistent with L_1).

3. LSH for Cosine Similarity

- Uses **SimHash** (Random Projections). The hash is determined by the sign of the dot product.

Product Quantization (PQ)

PQ is a lossy compression technique that allows for **Asymmetric Distance Computation (ADC)**.

- 1 **Decomposition:** A vector x of dimension D is split into m sub-vectors.
- 2 **Quantization:** For each sub-space, run k-means to find k^* centroids.
- 3 **Distance Estimation:** Approximate Euclidean distance by summing distances to centroids:

$$d(q, x) \approx \sum_{i=1}^m d(q_i, q^*(x_i)) \quad (7)$$

- This reduces memory usage from floats to small integer indices.

Hierarchical Navigable Small World (HNSW)

- **Top layers:** Contain only a few long-range edges for fast "coarse" navigation (logarithmic complexity).
- **Bottom layers:** Contain dense connections for "fine-grained" local search.
- Search starts at the top and greedily descends to the base layer.

Evaluating ANN: Recall@k

- Measured using **Recall**, comparing ANN results (R_{ANN}) with true k-NN results (R_{true}):

$$\text{Recall@}k = \frac{|R_{ANN} \cap R_{true}|}{k} \quad (8)$$

- Recall of 0.9 means 90% of nearest neighbors found are the true nearest neighbors.